University of Pisa

Parallel and Distributed Systems

Optimization of Partitioned Hash Join with Duplicates

Module 3 - Report

Gianluca Panzani

May 16, 2026

Contents

1 Implementation

1.1 Executable variants

The repository contains one sequential baseline and six OpenMP executables which are compared each other. There are three kind of OpenMP families based on the type of parallelization strategy: `loop`, `task` and `taskloop`. The last one is added only as an experimental comparison, to see whether OpenMP's taskloop construct behaves closer to manual tasks or to ordinary loop scheduling. Each family also has a `_wb` version that adds workload balancing on the join phase. All versions use the same flat open-addressing count table in the local build/probe phase, so the comparison is mostly about the parallel execution model and scheduling policy, not about a different join algorithm. So at the end we have the following seven executables: `hashjoin_seq`, `hashjoin_omp_loop`, `hashjoin_omp_loop_wb`, `hashjoin_omp_task`, `hashjoin_omp_task_wb`, `hashjoin_omp_taskloop` and `hashjoin_omp_taskloop_wb`.

1.2 Workflow

The implementation workflow strategy is the following:

- implement the OpenMP variants,
- run correctness checks,
- execute an initial larger grid search for each OpenMP family with a uniform dataset,
- analyze the generated CSV files with the statistics notebooks,
- reduce the grid to the most promising parameter regions,
- run the reduced campaign on uniform and skewed datasets to analyze how the different parallelization strategies perform on the different datasets.

1.3 Parameters

The benchmark scripts run Slurm jobs on one node of the `spmcluster` and append one CSV row per configuration in `src/results/`. The fixed parameters of the main campaign are $N_R = N_S = 50\,000\,000$, $P = 256$, `seed=13`, `max_key=1 000 000` and block size 32768 in the reduced grid.

The parameters of the final grids were not chosen directly. Initially i've created a larger grid for each family type and then, analyzing the relative notebook files, i've chosen the most promising values because of the huge amount of combinations. The grid of the `loop` family explores OpenMP schedules (`static`, `dynamic`, `guided`), chunk sizes, partition/join thread counts and partition block sizes. The grid of the `task` one explores the number of input blocks per partition task, the number of partitions per join task, offset-task granularity and thread counts. And the grid of the experimental `taskloop` family explores the same decomposition through `taskloop` grainsizes.

The reduced grid used for the final results keeps the most stable choices found by those statistics files: guided scheduling for loop variants, explicit task grains in the range 2–4 for the task variants, taskloop grains in the range 2–8 for the experimental taskloop variants, 32 join threads for each family, and 32 or 64 partition threads.

Three datasets are measured. The uniform dataset is the same pseudo-random generation scheme used in the previous module. The skewed datasets are generated with the format `skewed_R_H` with: R% percentage of records sampled from keys that map to the first H% of partitions. In the final grids i've used `skewed_90_5` and `skewed_90_1`, which concentrate 90% of the records respectively into about 13 and 3 hot partitions.

2 Parallelization Strategy

2.1 Partition phase

In the loop versions, the available work is exposed as regular OpenMP loops and the runtime scheduling policy decides how iterations are assigned to threads. This keeps the code compact and makes schedule tuning simple: the same executable can test `static`, `dynamic` and `guided` behavior through `schedule(runtime)`. In practice this strategy is well suited when the work units are regular and numerous, because the OpenMP loop scheduler has low overhead and the tuning space is mostly reduced to schedule, chunk size and thread count.

The task versions use a more explicit decomposition. Instead of relying only on loop iteration scheduling, they create OpenMP tasks over groups of blocks and partition ranges. This gives more control over the granularity of the work submitted to the runtime, but it also introduces a more delicate tradeoff: small tasks improve balancing opportunities but increase task creation overhead, while large tasks reduce overhead but behave closer to coarse loop chunks. For this reason the task grid includes task-grain parameters in addition to the thread counts.

The taskloop version preserves the loop-shaped formulation but asks OpenMP to generate tasks from it, using `taskloop grainsize(...)`. It is simpler than manually creating explicit tasks and, in the results, often behaves competitively; however, the comparison on which we are focusing on remains between the sequential version and the loop/task strategies.

2.2 Join phase

After partitioning, the join phase is naturally parallel because partition p of R only has to be joined with partition p of S. In other words, for each pair of partitions the join is independent.

The non-`_wb` loop version expresses this with an OpenMP `for` loop over the partition ids and combines the final `join_count`, `checksum1` and `checksum2` with an OpenMP `reduction`. The non-`_wb` task versions instead create OpenMP tasks for groups of partition joins and each task writes its result into a private `partial` slot.

The important distinction is between the non-`_wb` and the `_wb` executables. In the non-`_wb` versions, the join work is submitted to OpenMP in the natural partition-id order $0, \ldots, P-1$. This is simple and works well when the dataset is uniform, because partitions have similar sizes and each local join has roughly similar cost. With skewed datasets, however, some partitions contain many more records and duplicate matches. Then a raw partition-id order can create a tail effect: most threads finish their small partitions while some few threads remain busy on hot partitions. The `_wb` variants address this by keeping the same local hash-join logic, but changing the work list given to OpenMP. Before the join starts, the work of each partition is estimated as

$$|R_p| + |S_p|,$$

and a target amount of records per join thread is computed. Partitions are sorted from largest to smallest and large partitions are assigned more than one logical thread by splitting their S_p probe range into subranges. Each subrange builds a private count table over the corresponding R_p partition, probes only its assigned part of S_p, and writes into a private result slot; therefore no mutex is needed in the join hot path.

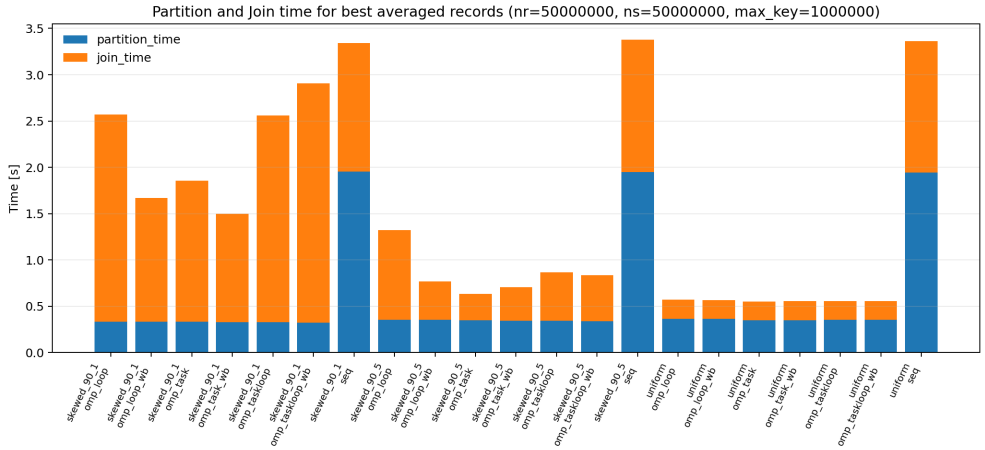

Figure 1: Comparison between partition and join times for each dataset type and executables combination.

3 Results Analysis

All results below can be found in the `statistics.ipynb` notebook whose analyzed data are picked from `src/results/*.csv`. The best Module 2 C++ threaded result was 0.507 s for `hashjoin_par_pj_wb_map`; the best averaged broad-grid OpenMP result here is 0.553 s, so the final OpenMP implementation is close in absolute time but does not clearly improve over the previous C++ threaded best case. This section covers the main Module 3 comparison: sequential baseline, OpenMP loop, OpenMP tasks, the extra taskloop experiment, uniform/skewed workloads, and the previous C++ threaded implementation.

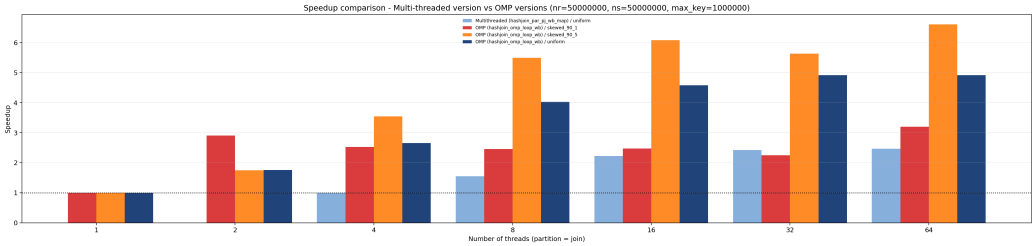

Figure 2: Speedup comparison between the previous multi-threaded `hashjoin_par_pj_wb_map` executable and the selected OpenMP workload-balanced loop implementation.

3.1 Uniform dataset

On the uniform workload, all complete OpenMP implementations are close once both partition and join are parallelized. The best averaged time is obtained by `hashjoin_omp_task` with 64 partition threads and 32 join threads: 0.553 s, corresponding to 6.08× speedup over the current sequential baseline.

The difference between the best loop, task and taskloop versions is small; this means that, for uniform partitions, OpenMP tasking does not add much by itself. The dominant gains come from parallelizing the join and from keeping the local hash table cache-friendly. Inside the required implementations, explicit tasks are slightly faster than loop scheduling on uniform data: `omp_task` reaches 0.553 s against 0.571 s for `omp_loop`. The extra taskloop variant remains very close to the best result and is also simpler to express than manual task creation, because OpenMP handles the task generation from the loop structure.

Variant	T_{part}	T_{join}	Best time [s]	Speedup
`omp_loop`	32	32	0.571	5.88×
`omp_loop_wb`	32	32	0.567	5.93×
`omp_task`	64	32	0.553	6.08×
`omp_task_wb`	64	32	0.558	6.03×
`omp_taskloop`	64	32	0.558	6.03×
`omp_taskloop_wb`	64	32	0.558	6.03×

Table 1: Best observed uniform-dataset configuration per OpenMP executable.

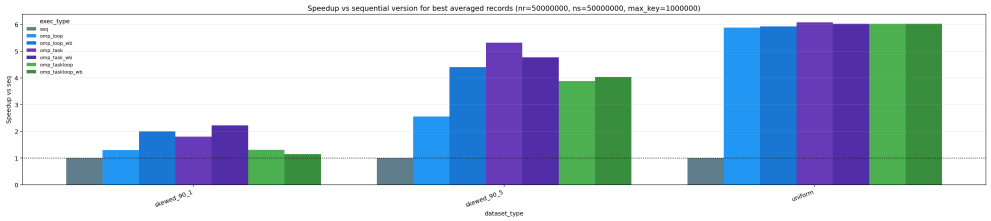

Figure 3: Best speedup against the sequential baseline for the current OpenMP campaign.

3.2 Skewed datasets

The skewed datasets change both load balance and the actual join workload. In the OpenMP skewed runs, the final match count rises from about $2.50 \cdot 10^9$ on uniform data to $4.06 \cdot 10^{10}$ on `skewed_90_5` and $1.74 \cdot 10^{11}$ on `skewed_90_1`. This happens because many records are concentrated into a small set of hot partitions and hot keys, increasing the probability that the same keys appear many times in both relations. Therefore the slowdown is not only scheduling overhead: the hot partitions also contain many more duplicate-key matches.

Dataset	Variant	T_{part}	T_{join}	Best time [s]	Join time [s]	Speedup
`skewed_90_5`	`omp_loop`	32	32	1.323	0.966	2.56×
`skewed_90_5`	`omp_loop_wb`	32	32	0.769	0.413	4.40×
`skewed_90_5`	`omp_task`	64	32	0.636	0.286	5.32×
`skewed_90_5`	`omp_task_wb`	64	32	0.710	0.362	4.77×
`skewed_90_5`	`omp_taskloop`	64	32	0.871	0.527	3.88×
`skewed_90_5`	`omp_taskloop_wb`	64	32	0.839	0.497	4.03×
`skewed_90_1`	`omp_loop`	32	32	2.573	2.236	1.30×
`skewed_90_1`	`omp_loop_wb`	32	32	1.670	1.334	2.00×
`skewed_90_1`	`omp_task`	64	32	1.856	1.521	1.80×
`skewed_90_1`	`omp_task_wb`	64	32	1.501	1.170	2.23×
`skewed_90_1`	`omp_taskloop`	64	32	2.561	2.234	1.31×
`skewed_90_1`	`omp_taskloop_wb`	64	32	2.912	2.586	1.15×

Table 2: Best observed skewed-dataset result for each OpenMP executable, using the sequential row with the same dataset label as baseline.

The workload-balanced loop version has the higher improvement with skewness, but is anyway slower than the task version. For `skewed_90_5`, the loop version improves from 1.323 s to 0.769 s when workload balancing is enabled. For `skewed_90_1`, the same change improves from 2.573 s to 1.670 s. The strongest skew still remains slower because splitting the probe side of a hot partition helps load balance but also repeats the R_p build work for each subrange.

Explicit tasks give the best required result on both skewed datasets in the current averaged campaign: `omp_task` reaches 0.636 s on `skewed_90_5`, while `omp_task_wb` reaches 1.501 s on `skewed_90_1`. The extra taskloop implementation remains useful experimentally, but its workload-balanced version is mixed in these runs.

4 Scaling analysis

Strong and weak scalability are measured only with the selected workload-balanced loop implementation, using the best fixed parameters found in the reduced grid. The loop version is used because it has the more balanced and expected behavior.

Both scaling files fix $P = 256$, `seed=13`, `max_key=1 000 000`, guided partition and join schedules, partition/join chunk size 8, and partition block size 32768. In `strong_scaling.cpp`, $N_R = N_S = 50\,000\,000$ is also fixed, while the only non-fixed parameter is the thread count, using the same value for partition and join phases. In `weak_scaling.cpp`, both $N_R = N_S$ and the thread count vary together, again with the same thread count for partition and join phases; their concrete values are reported in the plots below.

4.1 Weak scalability

Weak scaling worsens as the skew becomes stronger. On uniform data, increasing N together with the number of threads is still useful: the scaled speedup reaches about 3.17× at 16 threads and remains close to 3.02× at 32 threads. With moderate skew, `skewed_90_5`, the curve is lower and peaks earlier, at about 2.36× with 8 threads, then stays near 2.1× at 16 and 32 threads. With the strongest skew, `skewed_90_1`, the 2-thread case is almost ideal, but the curve then decreases to about 1.30× at 32 threads, because increasing N also increases the hot partitions and duplicate-key work that cannot be spread as evenly.

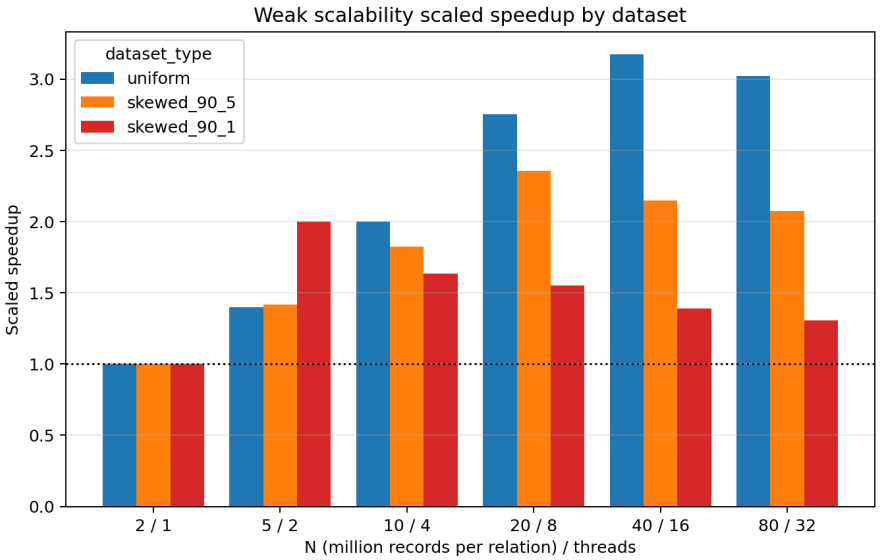

Figure 4: Weak-scaling scaled speedup for uniform and skewed datasets.

4.2 Strong scalability

Strong scaling shows a different effect of skew because the input size is fixed and the baseline is the 1-thread execution of each dataset. Uniform data improves regularly up to about 4.92× at 32 threads and then saturates at 64 threads. The moderate-skew case, `skewed_90_5`, has a slower 1-thread baseline and still contains enough work to split across threads, so its relative speedup is higher, reaching about 6.48× at 64 threads. The strongest skew, `skewed_90_1`, is not monotonic: it reaches about 3.13× at 2 threads, then stays below 3× for the larger thread counts. This shows that, when only a few hot partitions dominate the join, adding threads gives limited benefit even with workload balancing.

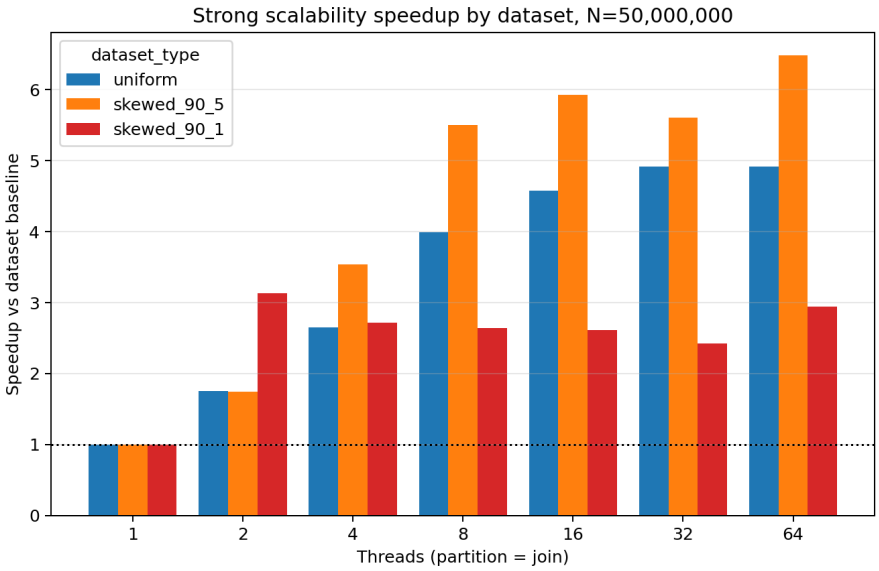

Figure 5: Strong-scaling speedup for the dedicated workload-balanced OpenMP executable.

5 Correctness Validation

Correctness is checked at two levels. For small inputs, $N_R \leq 500$ and $N_S \leq 500$, each executable runs the naive $O(N_R N_S)$ verifier and compares its `join_count`, `checksum1` and `checksum2` with the optimized result. If the comparison succeeds, the executable prints and records `verified=true`; if it fails, it prints `verified=false` and exits with an error. The checker therefore treats `verified=true` outputs as already validated by the exact naive oracle.

For benchmark-size inputs, where the naive verifier is too expensive, `checker.cpp` scans the Slurm output files and compares `join_count`, `checksum1` and `checksum2` for each executable and dataset type.

6 Conclusions

The OpenMP implementations preserve the previous algorithm and make the scheduling trade-offs clearer. On uniform data, loop, task and taskloop variants perform similarly once the join phase is parallelized; tasking does not provide a large advantage because partitions are already well balanced. On skewed data, workload-aware join decomposition is the decisive change for the loop family, while explicit tasks give the best required results in the current averaged campaign. The strongest skew still exposes a limitation: splitting a hot partition's probe range improves balance, but the repeated private build phase can become expensive.

The scalability results confirm the same trend: uniform data scales more regularly, moderate skew can still exploit extra threads, while the strongest skew quickly becomes limited by hot partitions and duplicate-key work.

Compared with the best C++ threaded result from Module 2, the best OpenMP result is close but slightly slower in the broad campaign. The main benefit of this module is therefore not a new absolute minimum, but a clearer understanding of which OpenMP constructs are sufficient and how join-work decomposition affects uniform and skewed workloads.